



11.4. Red-Black Trees

Balance Through Colors, Efficiency Through Rules

Previous Section:

 [11.3. AVL Trees](#)

Contents:

[1. Properties of a Red-Black Tree](#)

[1.1. Correct Red-Black Tree Example](#)

[1.2. Incorrect Red-Black Tree Example](#)

[2. Implementation of Red-Black Tree](#)

[2.1. Node Structure](#)

[2.2. Constructor and Sentinel Node](#)

[2.3. Traversal Operations](#)

[2.4. Searching](#)

[2.5. Insertion](#)

[2.6. Fixing Insertions](#)

[2.7. Deletion](#)

[2.8. Fixing Deletions](#)

[2.9. Tree Transplant](#)

[2.10. Rotation Operations](#)

[2.11. Utility Functions](#)

[2.12. Key Methods to Remember](#)

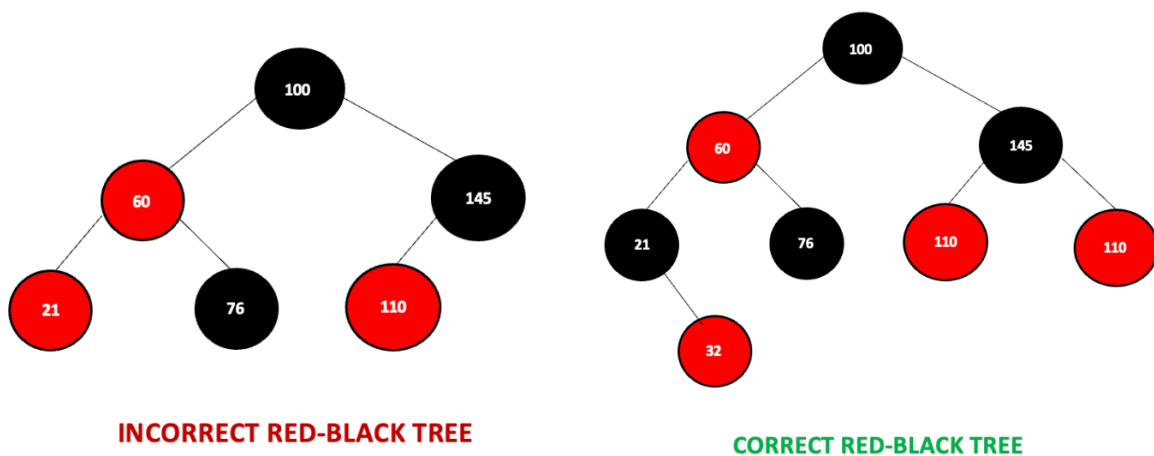
[Summary](#)

[References](#)

After studying AVL Trees, I might start wondering: ***“Do all self-balancing trees maintain balance by strictly controlling subtree heights?”***

Interestingly, the answer is no. AVL Trees maintain a very strict balance condition. Every node must have a balance factor between -1 and 1. This keeps the tree extremely balanced, but it also means insertions and deletions may trigger multiple rotations.

Red-Black Trees take a different approach. Instead of storing height information and enforcing strict balance, a Red-Black Tree assigns a color to every node: **RED** or **BLACK**. These colors are not used for display purposes. They act as additional information that helps the tree maintain a roughly balanced structure.



Like AVL Trees and B-Trees, the goal remains exactly the same: **Keep the tree balanced. Prevent degeneration into a linked list. Maintain efficient searching. Maintain efficient insertion and deletion**

However, Red-Black Trees achieve this balance using a set of coloring rules rather than strict height constraints.

As a result, Red-Black Trees are usually **less balanced than AVL Trees**, but they require fewer rotations during insertion and deletion operations.

Because of this characteristic, Red-Black Trees are widely used in practical software systems, operating systems, language libraries, and database implementations where updates happen frequently.

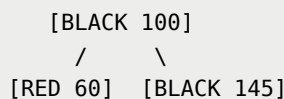
1. Properties of a Red-Black Tree

A Binary Search Tree becomes a valid Red-Black Tree only when it satisfies a specific set of rules. Think of these rules as the laws that every node must obey. If even one rule is violated, the tree is no longer considered a correct Red-Black Tree.

Property 1: Every Node Has a Color

Every node must be assigned exactly one color: **RED** or **BLACK**. No node is allowed to have an undefined color.

For example:

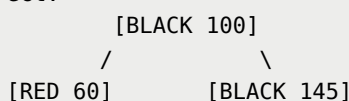


Every node has been assigned either red or black.

Property 2: The Root Node Must Always Be Black

The root is the topmost node of the tree. A Red-Black Tree always requires the root node to be black.

Correct:



Incorrect:



Since the root is red, the second tree violates the Red-Black Tree rules.

Property 3: A Red Node Cannot Have a Red Child

This is one of the most important properties. Whenever I encounter a red node, all of its children must be black.

Correct:

```
      [BLACK 100]
      /
    [RED 60]
    /
  [BLACK 21]
```

Notice: RED 60 → BLACK 21. This is valid.

Incorrect:

```
      [BLACK 100]
      /
    [RED 60]
    /
  [RED 21]
```

Notice: **RED 60 → RED 21**

This immediately violates the Red-Black Tree property.

We often call this situation a **Red-Red Violation**. Whenever insertion creates a Red-Red violation, the tree must perform recoloring or rotations to restore balance.

Property 4: Every Leaf Node Is Black

In Red-Black Trees, leaf nodes are treated slightly differently from ordinary Binary Search Trees. Conceptually, every missing child is represented by a special NIL node.

For example:

```
      [BLACK 100]
      /
    [RED 60]
```

Can be visualized as:

```
      [BLACK 100]
      /      \
    [RED 60]  [BLACK NIL]
```

```

      /   \
[BLACK NIL] [BLACK NIL]

```

All NIL nodes are always black. Therefore, **every leaf node in a Red-Black Tree is black**. This property helps maintain consistent black-node counts throughout the tree.

Property 5: Every Path Must Contain the Same Number of Black Nodes

This is the rule that truly gives Red-Black Trees their balancing power.

Consider the following tree:

```

          [BLACK 100]
        /      \
    [RED 60]    [BLACK 145]
    /  \      /  \
[BLACK 21] [BLACK 76] [RED 110] [RED 180]

```

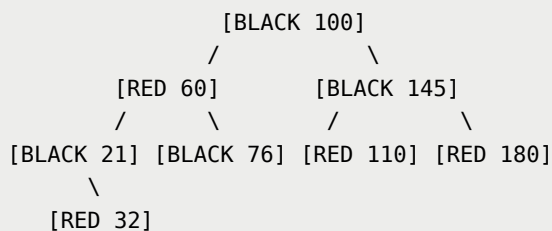
Let's count the black nodes on different paths.

- **Path 1:** 100 → 60 → 21
 - Black nodes: 100 and 21
 - Count: 2
- **Path 2:** 100 → 60 → 76
 - Black nodes: 100 and 76
 - Count: 2
- **Path 3:** 100 → 145 → 110
 - Black nodes: 100 and 145
 - Count: 2
- **Path 4:** 100 → 145 → 180
 - Black nodes: 100 and 145
 - Count: 2

Every path contains exactly: **2 black nodes**. Therefore, the tree satisfies the Black Height Property.

1.1. Correct Red-Black Tree Example

Consider the following structure:



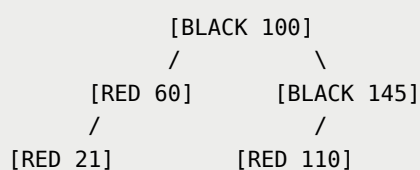
Let us verify the rules:

- Every node has a color ✓.
- Root node is black ✓.
- No red node has a red child ✓.
- Every NIL leaf is black ✓.
- Every root-to-leaf path contains the same number of black nodes ✓.

Since all properties are satisfied, this is a valid Red-Black Tree.

1.2. Incorrect Red-Black Tree Example

Now consider:



Immediately we can see: **RED 60 → RED 21**

A red node has a red child. This violates Property 3. Therefore, regardless of the other properties, the tree is not a valid Red-Black Tree.

The fascinating part of Red-Black Trees is that they do not try to make every subtree perfectly balanced like AVL Trees. Instead, they enforce these coloring rules, and those rules indirectly keep the tree balanced enough to guarantee efficient operations while reducing the number of rotations required during updates.

2. Implementation of Red-Black Tree

Now that we understand the properties of a Red-Black Tree, the next step is implementing one in Python.

At first glance, the implementation looks similar to a Binary Search Tree. Nodes still contain left and right pointers, and searching still follows the BST property. However, Red-Black Trees introduce several additional concepts:

- Every node stores a color.
- Every node stores a parent pointer.
- A special NIL node (called `TNULL`) is used instead of Python's `None`.
- Insertions and deletions require fixing operations to maintain Red-Black properties.
- Rotations are used together with recoloring.

Because of these extra requirements, the implementation is considerably more complex than both BST and AVL Tree implementations.

2.1. Node Structure

Every Red-Black Tree node stores:

```
class Node:
    def __init__(self, data, color=1):
        self.data = data
        self.color = color
        self.left = None
        self.right = None
        self.parent = None
```

The constructor creates a new node. Each node stores the actual value (`data`), node color, 0 for red, 1 for black (`color`), pointers to child nodes (`left` , `right`), pointers to parent nodes (`parent`).

Unlike AVL Trees, there is no height field. Red-Black Trees maintain balance using colors rather than subtree heights.

2.2. Constructor and Sentinel Node

The constructor creates the tree and initializes a special NIL node.

```
def __init__(self):
    self.TNULL = Node(0, 0)

    self.TNULL.color = 0
    self.TNULL.left = None
    self.TNULL.right = None

    self.root = self.TNULL
```

- The most important feature here is `self.TNULL` . This is called a **sentinel node**. Instead of storing `None` for missing children, Red-Black Trees use a special node.

Instead of representing:

```
    50
   /
  30
```

With several `None` references, the implementation actually stores:

```
      50
     / \
    30  NIL
   / \
  NIL NIL
```

Using a sentinel node simplifies insertion and deletion algorithms.

2.3. Traversal Operations

Traversal methods work almost exactly like Binary Search Trees.

- **Pre-Order Traversal**

```
def pre_order_traversal(self, node):  
    if node != self.TNULL:  
        print(node.data, end=' ')  
        self.pre_order_traversal(node.left)  
        self.pre_order_traversal(node.right)
```

- **In-Order Traversal**

```
def in_order_traversal(self, node):  
    if node != self.TNULL:  
        self.in_order_traversal(node.left)  
        print(node.data, end=' ')  
        self.in_order_traversal(node.right)
```

- **Post-Order Traversal**

```
def post_order_traversal(self, node):  
    if node != self.TNULL:  
        self.post_order_traversal(node.left)  
        self.post_order_traversal(node.right)  
        print(node.data, end=' ')
```

2.4. Searching

```
def search_tree(self, node, key):
    if node == self.TNULL or key == node.data:
        return node

    if key < node.data:
        self.search_tree(node.left, key)

    return self.search_tree(node.right, key)
```

Searching complexity is $O(\log n)$ on a balanced Red-Black Tree.

2.5. Insertion

Insertion begins exactly like a Binary Search Tree. The difference comes after insertion. Every newly inserted node is colored red.

```
def insert_node(self, data):

    node = Node(data)

    node.parent = None
    node.data = data
    node.left = self.TNULL
    node.right = self.TNULL

    node.color = 0

    y = None
    x = self.root

    while x != self.TNULL:

        y = x

        if node.data < x.data:
            x = x.left
```

```

        else:
            x = x.right

    node.parent = y

    if y == None:
        self.root = node

    elif node.data < y.data:
        y.left = node

    else:
        y.right = node

    if node.parent == None:
        node.color = 1
        return

    if node.parent.parent == None:
        return

    self.fix_insert(node)

```

- Step 1: Insert the node exactly like a BST.
- Step 2: Color the new node red with `node.color = 0`
- Step 3: Check whether any Red-Black Tree property is violated.
- Step 4: Call `self.fix_insert(node)` to restore balance.

2.6. Fixing Insertions

```

def fix_insert(self, k): # Fix the tree after insertion
    while k.parent.color == 0:
        if k.parent == k.parent.parent.right:
            u = k.parent.parent.left # uncle
            if u.color == 0: # case 3.1

```

```

        u.color = 1
        k.parent.color = 1
        k.parent.parent.color = 0
        k = k.parent.parent
    else:
        if k == k.parent.left: # case 3.2.2
            k = k.parent
            self.right_rotate(k)
        # case 3.2.1
        k.parent.color = 1
        k.parent.parent.color = 0
        self.left_rotate(k.parent.parent)
    else:
        u = k.parent.parent.right # uncle

        if u.color == 0: # mirror case 3.1
            u.color = 1
            k.parent.color = 1
            k.parent.parent.color = 0
            k = k.parent.parent
        else:
            if k == k.parent.right: # mirror case
                k = k.parent
                self.left_rotate(k)
            # mirror case 3.2.1
            k.parent.color = 1
            k.parent.parent.color = 0
            self.right_rotate(k.parent.parent)
    if k == self.root:
        break
    self.root.color = 1 # root is always black

```

3.2.2

This is the most important insertion method. Its purpose is to repair: Red-Red Violations. For example:

Before insertion:

BLACK 100

```

    /
   RED 60

Insert: RED 21
Result:

   BLACK 100
    /
   RED 60
    /
  RED 21

```

Violation: **RED** → **RED**. The tree becomes invalid. The fix operation may perform recoloring, left Rotation, right rotation, combination rotations, until all Red-Black properties are restored.

Finally: `self.root.color = 1` guarantees that the root remains black.

2.7. Deletion

Deletion is significantly more complicated than insertion.

```

def delete_node(self, data):
    node = self.root
    z = self.TNULL
    while node != self.TNULL:
        if node.data == data:
            z = node

            if node.data <= data:
                node = node.right
            else:
                node = node.left

    if z == self.TNULL:
        print("Couldn't find key in the tree")
        return

    y = z
    y_original_color = y.color
    if z.left == self.TNULL:

```

```

        x = z.right
        self.rb_transplant(z, z.right)
    elif (z.right == self.TNULL):
        x = z.left
        self.rb_transplant(z, z.left)
    else:
        y = self.get_minimum(z.right)
        y_original_color = y.color
        x = y.right
        if y.parent == z:
            x.parent = y
        else:
            self.rb_transplant(y, y.right)
            y.right = z.right
            y.right.parent = y

        self.rb_transplant(z, y)
        y.left = z.left
        y.left.parent = y
        y.color = z.color
    if y_original_color == 1:
        self.fix_delete(x)

```

Deletion first behaves exactly like a BST. Three cases are handled:

- **Case 1 - Leaf Node:** Simply remove it.
- **Case 2 - One Child:** Child replaces parent.
- **Case 3 - Two Children:** Replace with the successor

After deletion, Red-Black properties may be violated. Therefore

`self.fix_delete(x)` may be called.

2.8. Fixing Deletions

```

def fix_delete(self, x): # Fix the tree after deletion
    while x != self.root and x.color == 1:
        if x == x.parent.left:

```

```

s = x.parent.right
if s.color == 0:
    s.color = 1
    x.parent.color = 0
    self.left_rotate(x.parent)
    s = x.parent.right

if s.left.color == 1 and s.right.color ==
1:

    s.color = 0
    x = x.parent
else:
    if s.right.color == 1:
        s.left.color = 1
        s.color = 0
        self.right_rotate(s)
        s = x.parent.right

    s.color = x.parent.color
    x.parent.color = 1
    s.right.color = 1
    self.left_rotate(x.parent)
    x = self.root
else:
    s = x.parent.left
    if s.color == 0:
        s.color = 1
        x.parent.color = 0
        self.right_rotate(x.parent)
        s = x.parent.left

    if s.right.color == 1 and s.left.color ==
1:

        s.color = 0
        x = x.parent
    else:
        if s.left.color == 1:
            s.right.color = 1

```

```

        s.color = 0
        self.left_rotate(s)
        s = x.parent.left

        s.color = x.parent.color
        x.parent.color = 1
        s.left.color = 1
        self.right_rotate(x.parent)
        x = self.root
    x.color = 1

```

Deletion may break: Black-height property; Root property; Coloring property

The fix operation restores them. It uses recoloring, left rotation, or right rotation depending on the color of the parent, the sibling, and the sibling's children. This is considered the most complex operation in a Red-Black Tree.

2.9. Tree Transplant

```

def rb_transplant(self, u, v):
    if u.parent == None:
        self.root = v
    elif u == u.parent.left:
        u.parent.left = v
    else:
        u.parent.right = v

    v.parent = u.parent

```

This helper function replaces one subtree with another. For instance, node 70 is the right child of 50, we can replace that node with 80.

This operation is heavily used during deletion.

2.10. Rotation Operations

Rotations in Red-Black Trees are identical to those in AVL Trees.

The difference is that AVL Trees rotate because of height imbalance, while Red-Black Trees rotate because color properties are violated.

- **Left Rotation**

```
def left_rotate(self, x):  
  
    y = x.right  
  
    x.right = y.left  
  
    if y.left != self.TNULL:  
        y.left.parent = x  
  
    y.parent = x.parent  
  
    if x.parent == None:  
        self.root = y  
  
    elif x == x.parent.left:  
        x.parent.left = y  
  
    else:  
        x.parent.right = y  
  
    y.left = x  
  
    x.parent = y
```

- **Right Rotation**

```
def right_rotate(self, x):  
  
    y = x.left  
  
    x.left = y.right
```

```

if y.right != self.TNULL:
    y.right.parent = x

y.parent = x.parent

if x.parent == None:
    self.root = y

elif x == x.parent.right:
    x.parent.right = y

else:
    x.parent.left = y

y.right = x

x.parent = y

```

2.11. Utility Functions

Several helper functions make insertion and deletion operations easier.

- **Root:** Returns the root node.

```

def get_root(self):
    return self.root

```

- **Maximum:** Returns the largest value.

```

def get_maximum(self, node):
    while node.right != self.TNULL:
        node = node.right
    return node

```

- **Minimum:** Returns the smallest value.

```
def get_minimum(self, node):  
    while node.left != self.TNULL:  
        node = node.left  
    return node
```

- **Predecessor:** Returns the largest node smaller than the current node.

```
def get_predecessor(self, node):  
    if node.left != self.TNULL:  
        return self.get_maximum(node.left)  
  
    y = node.parent  
    while y != None and node == y.left:  
        node = y  
        y = y.parent  
    return y
```

- **Successor:** Returns the smallest node larger than the current node.

```
def get_successor(self, node):  
    if node.right != self.TNULL:  
        return self.get_minimum(node.right)  
  
    y = node.parent  
    while y != None and node == y.right:  
        node = y  
        y = y.parent  
    return y
```

Below is the full implementation of Red-Black Tree:

red_black_tree.py

2.12. Key Methods to Remember

When studying Red-Black Trees, the most important methods are:

```
insert_node()  
delete_node()  
fix_insert()  
fix_delete()  
left_rotate()  
right_rotate()
```

These methods are responsible for maintaining all Red-Black Tree properties. Without them, the structure would simply behave like an ordinary Binary Search Tree.

Summary

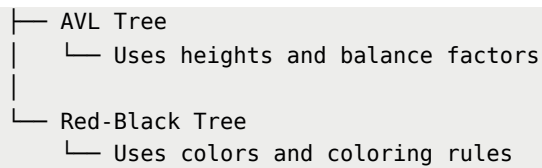
A Red-Black Tree is a self-balancing Binary Search Tree that uses node colors and a set of balancing rules to maintain efficient performance.

Unlike AVL Trees, a Red-Black Tree:

- Does not store node heights.
- Does not use balance factors.
- Does not attempt to keep every subtree perfectly balanced.
- Uses colors, recoloring, and rotations to maintain approximate balance.

This means an AVL Tree and a Red-Black Tree belong to the same family of self-balancing Binary Search Trees, but they are completely different data structures.

```
Binary Search Tree  
|
```



A Red-Black Tree is **not an AVL Tree**, nor is it a variation of AVL Tree. It is an independent self-balancing tree with its own balancing strategy.

Because it generally performs fewer rotations during updates, Red-Black Trees are often preferred in systems with frequent insertions and deletions, while AVL Trees are typically preferred when search operations dominate and stricter balancing is desired.

References

https://drive.google.com/file/d/1tOqiD1hwN4q_ZbZHeOp-e3oeJfgTEdDq/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/introduction-to-red-black-tree/>

<https://www.youtube.com/watch?v=TIIfQOdeFy0Y>

<https://www.youtube.com/watch?v=GmnODOF6Y8s>

Next Section:

11.5. B-Trees